Attendance Monitoring

COMP2007: Operating Systems & Concurrency
Week 6 – 12:00pm Wednesday – 30 October 2024

valid for 75 minutes from 11:45am
generated 2024-09-20 14:34

Operating Systems and Concurrency

Memory Management 1

COMP2007

Geert De Maere
(Dan Marsden)
{Geert.DeMaere,Dan.Marsden}@Nottingham.ac.uk

University Of Nottingham
United Kingdom

2024

Remember

Subjects We Will Discuss

Subject	#Lectures	By
Introduction to operating systems/computer design	3	DM
Processes, process scheduling, threading, ...	4	DM
Concurrency (deadlocks)	6	DM
Memory management, swapping, virtual memory, ...	6	GDM
File Systems, file structures, management, ...	5	GDM
Q&A	1	GDM

Table: Course structure

Goals for Today

Overview

1. **Introduction** to memory management
2. **Modelling** of multi-programming
3. Memory management based on **fixed partitioning**

Memory Management

Memory Hierarchies

- Computers typically have memory hierarchies:
  - **Registers**, L1/L2/L3 **cache**
  - **Main memory** (RAM)
  - **Disks**
- "**Higher memory**" is faster, more expensive and volatile, "**lower memory**" is slower, cheaper, and non-volatile
- The operating system provides a **memory abstraction**
- Memory can be seen as one **linear array** of bytes/words

©University of Nottingham

Memory Management

OS Responsibilities

- **Allocate/deallocate** memory when requested by processes
- Keep track of **used/unused** memory
- **Distribute memory** between processes and simulate an **“infinitely" large** memory space
- **Control access** when multi-programming is applied
- **Transparently** move data from **memory** to **disk** and vice versa

©University of Nottingham

Memory Management

History of Memory Management

- Memory management has **evolved** over time
- **History repeats** itself:
  - Many of the **early ideas underpin** more **modern memory management** approaches (e.g. relocation)
  - Modern **consumer electronics** often require **less complex memory management** approaches

©University of Nottingham

Partitioning

Approaches: Contiguous vs. Non-Contiguous

Figure: contiguous

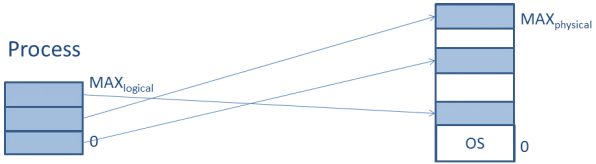


Figure: non-contiguous

Models

Approaches: Contiguous vs. Non-Contiguous

- **Contiguous memory management models** allocate memory in **one single block** without any **holes** or **gaps**
- **Non-contiguous memory management models**:
  - Allocate memory in **multiple blocks** (or **segments**)
  - May be **placed anywhere in physical memory** (i.e., not necessarily next to each other)

Partitioning

Contiguous Approaches

- **Mono-programming**: one single partition for user processes
- **Multi-programming** with **fixed partitions**
  - Fixed **equal** sized partitions
  - Fixed **non-equal** sized partitions
- **Multi-programming** with **dynamic partitions**

Mono-Programming

No Memory Abstraction

- Only **one single user process**, i.e. no multi-programming
  - A fixed region of memory is allocated to the **OS/kernel**
  - Remaining memory is reserved for a **single process**
  - This process has **direct access** to **physical memory**
- MS-DOS worked this way

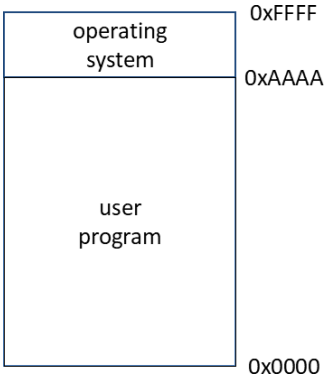


Figure: Mono-programming

Mono-Programming

No Memory Abstraction

- Only **one single user process**, i.e. no multi-programming
  - A fixed region of memory is allocated to the **OS/kernel**
  - Remaining memory is reserved for a **single process**
  - This process has **direct access** to **physical memory**
- MS-DOS worked this way

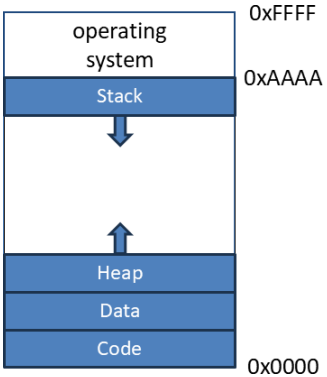


Figure: Mono-programming

Mono-Programming

No Memory Abstraction: Properties

- Every process is allocated **contiguous block of memory**
- **One process** is allocated the **entire remaining memory space**
- That process must always be located in the **same location**
- **No protection** between different user processes
- **Overlays** enable the programmer to use more memory than available
  - This is a burden on the programmer

Mono-Programming

No Memory Abstraction: Properties (Cont'ed)

- **Shortcomings** of mono-programming:
  - **Direct access to the physical memory** $\Rightarrow$ it may have **access to OS** memory
  - The operating system is a process $\Rightarrow$ so we have **two processes anyway**
  - **Multi-programming is expected** on modern machines
- Simulate multi-programming through:
  - Swapping: **write process image** to the disk and load a new one (**time consuming**)
  - Threads: **threads** within the same process
- Mono-programming is common in **embedded systems** and **modern consumer electronics**

©University of Nottingham

Mono-Programming

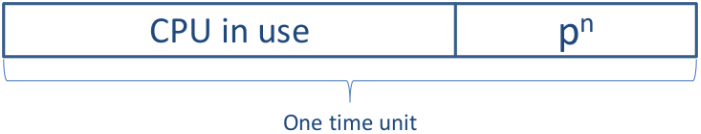
Simulating Multi-Programming

- Mono-programming results in **low utilisation** of resources (CPU, I/O devices, etc.)
  - Imagine a **disk** rotating at 7200 RPM, taking 4.2 ms to rotate half a track
  - Imagine a **CPU** running at 3.2 GHz (approx. 3.2×10^9 instructions per second)
  - I/O is slow, we are **missing out on** $3.2 \times 4.2 \times 10^6$ **instructions** (13.44m)!
- Assumption that **multi-programming** can **improve CPU utilisation**?

Multi-Programming

A Probabilistic Model

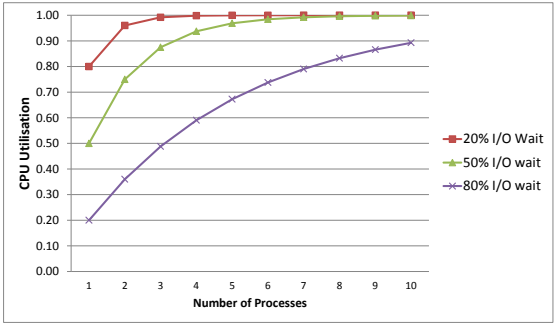
- A process spends p percent of its time **waiting for I/O**
- **CPU Utilisation** is calculated as 1 minus the time that all processes are waiting for I/O: e.g., $p = 0.9$ then CPU utilisation = 1 - p = 1 - 0.9 = 0.1
- If n processes are in memory, the probability that **all are waiting for I/O** simultaneously is p^n
- The **CPU utilisation** is given by $1 - p^n$



Multi-Programming

A Probabilistic Model

	I/O Ratio		
# Processes	0.2	0.5	0.8
1	0.80	0.50	0.20
2	0.96	0.75	0.36
3	0.99	0.88	0.49
4	1.00	0.94	0.59
5	1.00	0.97	0.67
6	1.00	0.98	0.74
7	1.00	0.99	0.79
8	1.00	1.00	0.83
9	1.00	1.00	0.87
10	1.00	1.00	0.89



CPU utilisation as a function of the I/O ratio and the number of processes

©University of Nottingham

Multi-Programming

A Probabilistic Model

- Assume that:
  - A computer has **1 MB** of memory
  - The **OS takes up 200KB**, leaving room for **four 200KB processes**
- Then:
  - If we have an **I/O wait time of 80%**, then we will achieve just under **60% CPU utilisation** ($1 - 0.8^4$)
  - If we add **1 MB of memory**, it would allows us to run another **five processes**
  - We can achieve about **87% CPU utilisation** ($1 - 0.8^9$)
  - If we add another **megabyte of memory** (fourteen processes) we will find that the CPU utilisation will increase to **about 96%** ($1 - 0.8^{14}$)

©University of Nottingham

Multi-Programming

A Probabilistic Model

Caveats:

- This model assumes that **all processes are independent**, this is not true
- More complex models could be built using **queueing theory**, but we can still use this simplistic model to **make approximate predictions**

Partitioning

Fixed Partitions of equal size

- **Multi-programming** improves **resource utilisation**
- Use **static equal sized contiguous partitions**
  - Any process can take **any** (large enough) **partition**
  - **Allocation** of processes to partitions is **trivial**
  - Very **little overhead** and **simple implementation**
  - The OS keeps a track of which partitions are being **used** and which are **free**

(a) Equal-size partitions

Figure: From Stallings

Partitioning

Fixed Partitions of equal size

- **Disadvantages** of **static equal-sized partitions**
  - **Low memory utilisation**
  - **excessive internal fragmentation**
  - Requires the use of **overlays** (burden on programmer)

(a) Equal-size partitions

Figure: From Stallings

Partitioning

Fixed Partitions of non-equal size

- Use **static <u>non</u>-equal sized contiguous partitions**
  - **Reduces internal fragmentation**
  - **Allocation** of processes must be **carefully considered**

(b) Unequal-size partitions

Figure: From Stallings

Partitioning

Fixed Partitions (Allocation Methods)

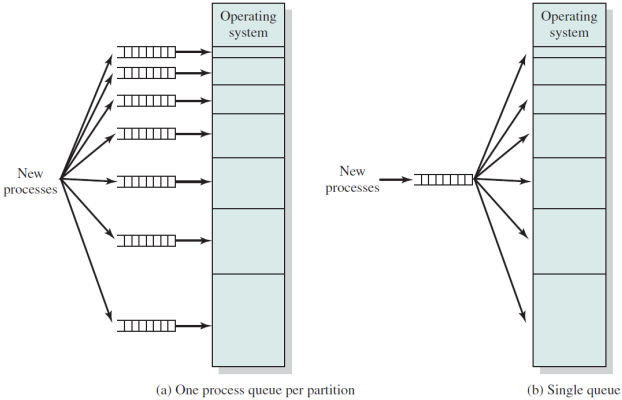


Figure: From Stallings

Partitioning

Fixed Partitions (Allocation Methods)

- One **private queue** per partition:
  - Assigns each process to the **smallest partition** that it would fit in
  - Reduces **internal fragmentation**
  - Can **reduce memory utilisation** and result in **starvation**
    - e.g., lots of small jobs result in unused large partitions
- A single **shared queue** for all partitions can allocate small processes to **large partitions** but results in **increased internal fragmentation**

©University of Nottingham

Test Your Understanding

- What other issue have we ignored **the issue**?
- How would you **resolve it**?

Test Your Understanding

- What other issue have we ignored **the issue**?
- How would you **resolve it**?
- Tip: the compiler allocates **memory addresses**

©University of Nottingham

Recap

Take-Home Message

- **Mono-programming** and **absolute addressing (no memory abstraction)**
- Why multi-programming: CPU utilisation modelling
- Memory management for **multi-programming**: fixed (non-)equal **partitions**